```
Output:
string(10) "some value"
```

Notice the double dollar symbol `$$name`. When this code runs, PHP will evaluate `$name`, retrieve its value (the string `'val'`), and use it to form a new variable `$val`. `$val` already exists, and so its value (the string `some value`) is then used as the data for the call to `var_dump`. Pretty simple stuff.

Be warned: using variable variables can present problems for debugging. Normally a simple lint (from the command line run `$ php -l` followed by the filename) will warn you if you use an uninitialized variable in your code, which risks emitting a potentially fatal error. If you use an IDE (integrated development environment) to write code, it should highlight issues like this.

However, a variable variable can only be resolved at runtime. Unless the code in question is very simple, it can be difficult to spot broken variable variables.

```
$val = 'some value';
$name = 'wrongName';
var_dump($$name);
```

```
Output:
Warning: Undefined variable $wrongName
```

If we run the php cli binary in "lint" (syntax checking) mode against the above code, it won't find any issues.

```
$ php -l broken_variable_variable.php
```

```
Output:
No syntax errors detected in broken_variable_variable.php
```

Variable variables have their place, but, like references, you should use them sparingly.

Summary

Hopefully you now have an understanding of the difference between data and information and that the same data (the raw byte values) can represent very different information depending on context. The way that PHP uses any item of data is determined by metadata called *type*, and data types can be implicitly and explicitly